

Lab 22 - Probes, Health and Self-Healing

Field	Value
Lab id	Lab 22
Day / module	Day 5 - Observability
Duration	45 minutes
Namespace	kcna-lab22
Maps to	LO6 Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting A6 Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework K7 Interactions among various IT components
Cluster	Single-node kind, Kubernetes v1.30+ (built and validated against v1.37.0)

How to read the expected-output blocks. Every command below is followed by the output you should see. Pod name suffixes, IP addresses, AGE columns and timestamps are generated at runtime and will differ on your cluster - the *shape*, the column headers, the status strings and the event reasons are what you are matching against. Nothing in this lab asks you to match a random hash.

1. Objective

By the end of this lab you will be able to:

1. Distinguish the three probe types by their *consequence*, not their syntax
 - a failing liveness probe restarts the container, a failing readiness probe removes the Pod from the Service's EndpointSlice, and a startup probe suspends both while a slow application boots.
2. Configure initialDelaySeconds, periodSeconds, timeoutSeconds and failureThreshold and calculate the resulting *failure budget* in seconds.
3. Prove that readiness gates traffic by watching a Service's EndpointSlice lose and regain an address while the container is never restarted.
4. Diagnose a restart loop caused by a liveness probe that is more aggressive than the application's real startup time, and repair it with a startup probe.

2. Prerequisites

* A running single-node kind cluster and a kubectl whose minor version is within one of the server. Verify:

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

- * Cluster-admin rights within your own namespace (the standard kind admin kubeconfig is sufficient).
- * You are in the lab directory. Every relative path below is from here:

```
cd courseware/labs/lab-22-probes-health-selfhealing
```

- * Images used, all pinned: nginx:1.27-alpine, busybox:1.36, alpine:3.20. Pull them once before you start so probe timings are not distorted by image download time on a slow connection:

```
kubectl run image-warm --rm -i --restart=Never --image=busybox:1.36 -n default -- true
```

```
pod "image-warm" deleted
```

3. Scenario

Meridian Freight Pte Ltd runs the Depot Portal, the web application its drivers use to check in shipments at the Tuas depot. Last Thursday the Depot Platform Squad shipped release 6.0.0 and opened incident INC-4417.

The rollout reported "2/2 available" within seconds, the dashboard went green, and drivers immediately started receiving HTTP 502 at the gate. The new replicas had been added to the Service's backend list before they could serve a single request, because the Deployment had no readiness probe - Kubernetes had no way to know the difference between *the process has started* and *the process can do work*.

The follow-up ticket DEP-1012 also flagged a second workload, the manifest-indexer, which spends about forty seconds rebuilding an in-memory shipment index at boot. An engineer "hardened" it by adding a tight liveness probe. It has been in CrashLoopBackOff ever since - and nothing is actually wrong with it.

You are the on-call platform engineer. Your job today is to make health signals mean something.

4. Step-by-step procedure

Part 1 - Create the namespace and the content ConfigMap

Step 1.1 - Create the lab namespace.

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab22 created
```

Step 1.2 - Confirm the Pod Security Admission labels landed. The lab runs under the baseline profile; nothing in this lab is privileged.

```
kubectl get namespace kcna-lab22 --show-labels
```

NAME	STATUS	AGE	LABELS
kcna-lab22	Active	4s	app.kubernetes.io/part-of=depot-portal,kcna.tertiaryinfotech.com/day=5,kcna.tertiaryinfotech.com/lab=lab-22,kubernetes.io/metadata.name=kcna-lab22,pod-security.kubernetes.io/enforce-version=latest,pod-security.kubernetes.io/enforce=baseline

Step 1.3 - Build the ConfigMap from the two HTML files in data/. These are the real probe targets: index.html is what liveness asks for, ready.html is what readiness asks for.

```
kubectl create configmap depot-portal-content \
  --from-file=index.html=data/depot-portal-content/index.html \
  --from-file=ready.html=data/depot-portal-content/ready.html \
  -n kcna-lab22
```

```
configmap/depot-portal-content created
```

Step 1.4 - Verify the keys.

```
kubectl get configmap depot-portal-content -n kcna-lab22 -o jsonpath='{.data}' | tr ',' '\n' | cut -c1-60
```

```
{"index.html":"<!doctype html>\n<!--\n  Meridian Freight Pt\n  \"ready.html\":\"READY depot-portal sg-tuas inbound\n# Meridia
```

Part 2 - Deploy depot-portal with readiness *and* liveness

Step 2.1 - Read the probe stanzas before you apply them. Open manifests/10-depot-portal.yaml and locate the two probes on the portal container. Note that they point at different paths on purpose:

Probe	Path	Budget before it acts	What happens when it fails
readinessProbe	/ready.html	$3 + (5 \times 2) = 13$ s	Pod address removed from the Service EndpointSlice. Container keeps running.
livenessProbe	/index.html	$10 + (10 \times 3) = 40$ s	Container is killed with SIGTERM and restarted. RESTARTS increments.

Step 2.2 - Apply the Deployment and its Service.

```
kubectl apply -f manifests/10-depot-portal.yaml
```

```
deployment.apps/depot-portal created
service/depot-portal created
```

Step 2.3 - Watch the Pods become Ready. The READY column is the *readiness* column: 1/1 means one of one containers is passing its readiness probe.

```
kubectl get pods -n kcna-lab22 -l app=depot-portal -w
```

NAME	READY	STATUS	RESTARTS	AGE
depot-portal-7c9b6d4f8b-2q6xl	0/1	Init:0/1	0	2s
depot-portal-7c9b6d4f8b-h4kzp	0/1	Init:0/1	0	2s
depot-portal-7c9b6d4f8b-2q6xl	0/1	PodInitializing	0	4s
depot-portal-7c9b6d4f8b-h4kzp	0/1	PodInitializing	0	4s
depot-portal-7c9b6d4f8b-2q6xl	0/1	Running	0	6s
depot-portal-7c9b6d4f8b-h4kzp	0/1	Running	0	6s
depot-portal-7c9b6d4f8b-2q6xl	1/1	Running	0	9s
depot-portal-7c9b6d4f8b-h4kzp	1/1	Running	0	9s

Press Ctrl-C to stop watching.

Read this carefully. There is a real gap between Running (~6 s) and

1/1 (~9 s). That gap is initialDelaySeconds: 3 plus one probe round trip.

Without a readiness probe there would be no gap at all - the Pod would be

declared Ready the instant it started running, which is exactly what caused

INC-4417.

Step 2.4 - Confirm the init container did its job.

```
kubectl logs -n kcna-lab22 -l app=depot-portal -c seed-content --tail=5 --prefix
```

```
[pod/depot-portal-7c9b6d4f8b-2q6xl/seed-content] [seed] webroot populated:
[pod/depot-portal-7c9b6d4f8b-2q6xl/seed-content] total 8
[pod/depot-portal-7c9b6d4f8b-2q6xl/seed-content] -rw-r--r--    1 root    root          383
Sep  5 06:12 index.html
[pod/depot-portal-7c9b6d4f8b-2q6xl/seed-content] -rw-r--r--    1 root    root          341
Sep  5 06:12 ready.html
```

Part 3 - Prove that readiness gates traffic

This is the heart of the lab. You are going to break readiness on one replica and watch that replica's IP leave the Service backend list.

Step 3.1 - Look at the Service's backends. Since Kubernetes v1.33 the v1 Endpoints object is deprecated; EndpointSlice is the current API and is what kube-proxy actually consumes.

```
kubectl get endpointslices -n kcna-lab22 -l kubernetes.io/service-name=depot-portal
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-portal-xk4t9	IPv4	80	10.244.0.14,10.244.0.15	62s

Two addresses - one per Ready replica.

Step 3.2 - Start the in-cluster client Pod. You will curl the Service from inside the cluster; nothing is exposed outside it.

```
kubectl apply -f manifests/40-probe-client.yaml
kubectl wait --for=condition=Ready pod/probe-client -n kcna-lab22 --timeout=60s
```

```
pod/probe-client created
pod/probe-client condition met
```

Step 3.3 - Confirm the Service answers. alpine:3.20 ships BusyBox wget, not curl, so use wget -qO-.

```
kubectl exec -n kcna-lab22 probe-client -- wget -qO- http://depot-portal.kcna-lab22.svc.cluster.local/ready.html
```

```
READY depot-portal sg-tuas inbound
# Meridian Freight Pte Ltd - Depot Portal readiness marker.
#
# This file is the READINESS target. The application would normally create it
# only after it has opened its database pool and warmed the shipment cache.
# In this lab you delete and restore it by hand so you can watch the Service
# EndpointSlice gain and lose this Pod, WITHOUT the container being restarted.
```

Step 3.4 - Capture the name of one replica into a shell variable. You will break only this one.

```
VICTIM=$(kubectl get pods -n kcna-lab22 -l app=depot-portal \
-o jsonpath='{.items[0].metadata.name}')
echo "victim = $VICTIM"
```

```
victim = depot-portal-7c9b6d4f8b-2q6x1
```

Step 3.5 - In a second terminal, start watching the EndpointSlice. Leave this running for the rest of Part 3.

```
kubectl get endpointslices -n kcna-lab22 -l kubernetes.io/service-name=depot-portal -w
```

Step 3.6 - Back in the first terminal, delete the readiness marker inside the victim container. Liveness still passes, because /index.html is untouched.

```
kubectl exec -n kcna-lab22 "$VICTIM" -c portal -- rm /usr/share/nginx/html/ready.html
```

```
(no output - success)
```

Step 3.7 - Within about 13 seconds, the watch in your second terminal drops an address:

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-portal-xk4t9	IPv4	80	10.244.0.14,10.244.0.15	3m
depot-portal-xk4t9	IPv4	80	10.244.0.15	3m12s

Step 3.8 - Confirm the Pod is 0/1 but that RESTARTS is still 0. This is the single most important observation in the lab.

```
kubectl get pods -n kcna-lab22 -l app=depot-portal
```

NAME	READY	STATUS	RESTARTS	AGE
depot-portal-7c9b6d4f8b-2q6x1	0/1	Running	0	3m20s
depot-portal-7c9b6d4f8b-h4kzp	1/1	Running	0	3m20s

Not Ready, not restarted, still Running. Readiness withdrew the Pod from

service; it did not punish the container.

Step 3.9 - Read the Pod conditions and the event the kubelet raised.

```
kubectl describe pod -n kcna-lab22 "$VICTIM" | sed -n '/^Conditions:\/,/^Volumes:/p'
```

Conditions:

Type	Status
PodReadyToStartContainers	True
Initialized	True
Ready	False
ContainersReady	False
PodScheduled	True

```
kubectl events -n kcna-lab22 --for pod/"$VICTIM" --types=Warning
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
10s (x3 over 20s)	Warning	Unhealthy	Pod/depot-portal-7c9b6d4f8b-2q6xl	Readiness probe failed: HTTP probe failed with statuscode: 404

Step 3.10 - Verify the Service still works. Traffic now only reaches the healthy replica - this is the graceful degradation that readiness buys you.

```
kubectl exec -n kcna-lab22 probe-client -- \
  sh -c 'for i in 1 2 3 4 5; do wget -qO- http://depot-portal.kcna-lab22.svc.cluster.local/
  ready.html | head -1; done'
```

```
READY depot-portal sg-tuas inbound
READY depot-portal sg-tuas inbound
READY depot-portal sg-tuas inbound
READY depot-portal sg-tuas inbound
READY depot-portal sg-tuas inbound
```

Five requests, five successes, zero errors - while one of your two replicas is broken. Compare that to INC-4417, where roughly half the requests returned 502.

Step 3.11 - Restore readiness by copying the marker back from the read-only ConfigMap mount.

```
kubectl exec -n kcna-lab22 "$VICTIM" -c portal -- \
  cp /etc/depot/content/ready.html /usr/share/nginx/html/ready.html
```

```
cp: can't stat '/etc/depot/content/ready.html': No such file or directory
command terminated with exit code 1
```

Expected obstacle - read it, do not skip it. The ConfigMap volume is

mounted into the init container only, not into portal. That is

deliberate: the runtime container should not be able to see its own seed data.

Recreate the file directly instead:

```
kubectl exec -n kcna-lab22 "$VICTIM" -c portal -- \
  sh -c 'echo "READY depot-portal sg-tuas inbound" > /usr/share/nginx/html/ready.html'
```

(no output - success)

Step 3.12 - Within ~5 seconds the address returns. Your second-terminal watch prints:

```
depot-portal-xk4t9    IPv4      80      10.244.0.15,10.244.0.14    4m10s
```

```
kubectl get pods -n kcna-lab22 -l app=depot-portal
```

NAME	READY	STATUS	RESTARTS	AGE
depot-portal-7c9b6d4f8b-2q6xl	1/1	Running	0	4m15s
depot-portal-7c9b6d4f8b-h4kzp	1/1	Running	0	4m15s

Stop the watch in the second terminal with Ctrl-C.

Step 3.13 - Record what you observed in data/probe-tuning-worksheet.csv, column observed_time_to_first_success_s, row depot-portal / readiness.

Part 4 - A slow starter done right: the startup probe

Step 4.1 - Apply the manifest-indexer. It sleeps 40 seconds before it listens, and its startupProbe grants a 120 second budget.

```
kubectl apply -f manifests/20-manifest-indexer-startup.yaml
```

```
deployment.apps/manifest-indexer created
service/manifest-indexer created
```

Step 4.2 - Watch it. It stays 0/1 Running for about 40 seconds and then flips to 1/1 - with zero restarts.

```
kubectl get pods -n kcna-lab22 -l app=manifest-indexer -w
```

NAME	READY	STATUS	RESTARTS	AGE
manifest-indexer-6d84f9c7b5-vn2tq	0/1	ContainerCreating	0	1s
manifest-indexer-6d84f9c7b5-vn2tq	0/1	Running	0	3s
manifest-indexer-6d84f9c7b5-vn2tq	1/1	Running	0	45s

Ctrl-C to stop.

Step 4.3 - Confirm no restarts occurred and the startup probe is what held the line.

```
kubectl describe pod -n kcna-lab22 -l app=manifest-indexer | grep -E 'Restart
Count|Liveness|Readiness|Startup'
```

```
Restart Count: 0
Liveness:      http-get http://:http/index.html delay=0s timeout=2s period=10s #success=1
#failure=3
Readiness:     http-get http://:http/index.html delay=0s timeout=2s period=5s #success=1
#failure=2
Startup:       http-get http://:http/index.html delay=0s timeout=2s period=5s #success=1
#failure=24
```

Step 4.4 - Read the startup events. You will see the startup probe failing repeatedly and then simply stopping - it is never run again once it succeeds.

```
kubectl events -n kcna-lab22 --for deployment/manifest-indexer 2>/dev/null | head -5
kubectl get events -n kcna-lab22 --field-selector reason=Unhealthy --sort-by=.lastTimestamp |
tail -3
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
55s	Warning	Unhealthy	Pod/manifest-indexer-6d84f9c7b5-vn2tq	Startup probe failed: Get "http://10.244.0.17:8080/index.html": dial tcp 10.244.0.17:8080: connect: connection refused

The mechanism. connection refused is the correct, expected failure while the process is still sleeping. The startup probe absorbs all of it. Liveness and readiness are not evaluated at all during this window - that is the entire point of the third probe type.

Part 5 - Verification

Run the bundled checks.

```
bash verification/checks.sh
```

See verification/expected-output.md for the full expected transcript and the pass criteria.

Part 6 - Failure injection (mandatory)

You will now reproduce ticket DEP-1012: the same slow application, but with the startup probe removed and liveness tuned aggressively.

Step 6.1 - Read the arithmetic before you apply it.

```
liveness budget = initialDelaySeconds 5 + (periodSeconds 5 × failureThreshold 2)
                = 15 seconds
real start time = 40 seconds
15 < 40 -> the container can never finish starting
```

Step 6.2 - Apply the broken Deployment.

```
kubectl apply -f manifests/30-manifest-indexer-outage.yaml
```

```
deployment.apps/manifest-indexer-outage created
```

Step 6.3 - Watch for about 90 seconds. RESTARTS climbs and the Pod enters CrashLoopBackOff.

```
kubectl get pods -n kcna-lab22 -l app=manifest-indexer-outage -w
```

NAME	READY	STATUS	RESTARTS	AGE
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	Running	0	5s
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	Running	1 (2s ago)	22s
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	Running	2 (1s ago)	41s
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	CrashLoopBackOff	2 (12s ago)	55s
manifest-indexer-outage-5f7c8b9d6-t8pnw	0/1	Running	3 (21s ago)	75s

Ctrl-C to stop.

Step 6.4 - Diagnose. The Warning Unhealthy + Killing pair is the signature of a liveness-induced restart loop.

```
BROKEN=$(kubectl get pods -n kcna-lab22 -l app=manifest-indexer-outage \
-o jsonpath='{.items[0].metadata.name}')
kubectl describe pod -n kcna-lab22 "$BROKEN" | tail -14
```

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	81s	default-scheduler	Successfully assigned kcna-lab22/manifest-indexer-outage-5f7c8b9d6-t8pnw to kind-control-plane
Normal	Pulled	22s (x4 over 80s)	kubelet	Container image "busybox:1.36" already present on machine
Normal	Created	22s (x4 over 80s)	kubelet	Created container: indexer
Normal	Started	22s (x4 over 80s)	kubelet	Started container indexer
Warning	Unhealthy	12s (x8 over 74s)	kubelet	Liveness probe failed: Get "http://10.244.0.18:8080/index.html": dial tcp 10.244.0.18:8080: connect: connection refused
Normal	Killing	12s (x4 over 66s)	kubelet	Container indexer failed liveness probe, will be restarted

Step 6.5 - Confirm the application itself is innocent. Its own log shows it was always mid-warm-up when it was killed - it never got to print the second line.

```
kubectl logs -n kcna-lab22 "$BROKEN" --previous
```

```
[indexer] 06:31:04 warming shipment manifest index...
```

Exactly one line. The index warm; serving on :8080 line never appears, because the kubelet killed the process at ~15 s and the sleep is 40 s.

Step 6.6 - Compare the two Deployments side by side to isolate the difference.

```
diff <(kubectl get deploy manifest-indexer -n kcna-lab22 -o jsonpath='{.spec.template.spec.containers[0].livenessProbe}{"\n"}{.spec.template.spec.containers[0].startupProbe}{"\n"}') \
    <(kubectl get deploy manifest-indexer-outage -n kcna-lab22 -o jsonpath='{.spec.template.spec.containers[0].livenessProbe}{"\n"}{.spec.template.spec.containers[0].startupProbe}{"\n"}')
```

```
1,2c1,2
< {"failureThreshold":3,"httpGet":{"path":"/index.html","port":"http","scheme":"HTTP"},
"periodSeconds":10,"successThreshold":1,"timeoutSeconds":2}
< {"failureThreshold":24,"httpGet":{"path":"/index.html","port":"http","scheme":"HTTP"},
"periodSeconds":5,"successThreshold":1,"timeoutSeconds":2}
---
> {"failureThreshold":2,"httpGet":{"path":"/index.html","port":"http","scheme":"HTTP"},
"initialDelaySeconds":5,"periodSeconds":5,"successThreshold":1,"timeoutSeconds":2}
>
```

The broken Deployment has no startup probe at all (empty second line), and a liveness failureThreshold of 2 instead of 3.

Step 6.7 - Repair it. Patch a startup probe onto the running Deployment rather than re-applying a file, so you see the rollout heal in place.

```
kubectl patch deployment manifest-indexer-outage -n kcna-lab22 --type=strategic -p '
spec:
  template:
    spec:
      containers:
      - name: indexer
        startupProbe:
          httpGet:
            path: /index.html
            port: http
          periodSeconds: 5
          timeoutSeconds: 2
          failureThreshold: 24
'
```

```
deployment.apps/manifest-indexer-outage patched
```

Step 6.8 - Confirm the heal. The new ReplicaSet's Pod reaches 1/1 with RESTARTS 0.

```
kubectl rollout status deployment/manifest-indexer-outage -n kcna-lab22 --timeout=150s
kubectl get pods -n kcna-lab22 -l app=manifest-indexer-outage
```

```
Waiting for deployment "manifest-indexer-outage" rollout to finish: 1 old replicas are pending
termination...
deployment "manifest-indexer-outage" successfully rolled out
```

NAME	READY	STATUS	RESTARTS	AGE
manifest-indexer-outage-6b5d497c84-w9jrx	1/1	Running	0	52s

Step 6.9 - Record the repair in data/probe-tuning-worksheet.csv, row manifest-indexer-outage, column verdict.

Step 6.10 - Remove only the injected Deployment.

```
kubectl delete deployment manifest-indexer-outage -n kcna-lab22
```

```
deployment.apps "manifest-indexer-outage" deleted
```

5. Verification

Run the scripted checks:

```
bash verification/checks.sh
```

You have completed the lab when all checks report PASS and you can answer these three questions without looking them up:

1. A Pod is 0/1 Running with RESTARTS 0. Which probe is failing, and is the Service still sending it traffic? *(Readiness; no.)*
2. A Pod is 0/1 and RESTARTS is climbing. Which probe is failing? *(Liveness - or a startup probe whose budget has been exhausted.)*
3. Your application takes 40 s to boot and you want liveness to catch a hang

within 30 s. Which probe do you add, and what must its budget exceed? *(A startup probe; its periodSeconds × failureThreshold must comfortably exceed 40 s.)*

6. Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix	
Pod stuck 0/1 Running, RESTARTS 0, never becomes Ready	Readiness probe path returns non-2xx/3xx, or the port name in the probe does not match a containerPort name	<code>kubectl describe pod <pod> -n kcna-lab22 \</code>	<code>grep -A2 Readiness</code> then <code>kubectl events -n kcna-lab22 --for pod/<pod></code>	Correct the path or the port name; probe port: must match the ports[].name on the same container
RESTARTS climbing, events show Liveness probe failed: connection refused shortly after start	Liveness budget is shorter than real startup time (the DEP-1012 bug)	<code>kubectl logs <pod> --previous</code> - the app log stops mid-boot	Add a startupProbe whose periodSeconds × failureThreshold exceeds worst-case boot time; leave liveness tight	
EndpointSlice shows <unset> under ENDPOINTS while Pods look Ready	Service selector does not match the Pod template labels	<code>kubectl get endpointslices -n kcna-lab22 -l kubernetes.io/service-name=depot-portal -o yaml</code> and compare with <code>kubectl get pods --show-labels</code>	Align spec.selector on the Service with the Pod labels	
Liveness probe failed: HTTP probe failed with statuscode: 404 immediately	Probe points at a file the image does not serve, or the webroot volume shadowed the image's default content	<code>kubectl exec <pod> -c portal -- ls /usr/share/nginx/html</code>	Point the probe at a path that exists, or seed the volume (this lab uses an init container for exactly this reason)	
Probes appear to "not run" during the first minute	A startupProbe is present and still failing - liveness/readiness are suspended by design	<code>kubectl describe pod <pod> \</code>	<code>grep Startup</code>	Nothing to fix; this is correct behaviour. Shorten the startup budget only if boot is genuinely faster
<code>kubectl exec ... -c portal</code> returns container portal is not valid for pod	Wrong container name, or you targeted the init container after it completed	<code>kubectl get pod <pod> -o jsonpath='{.spec.containers[*].name}'</code>	Use a name from that list; init containers cannot be exec-ed once they have terminated	

7. Cleanup

Delete only this lab's namespace. Everything you created lives inside it.

```
kubectl delete namespace kcna-lab22
```

```
namespace "kcna-lab22" deleted
```

```
kubectl get namespace kcna-lab22
```

```
Error from server (NotFound): namespaces "kcna-lab22" not found
```

This lab created no cluster-scoped objects and touched nothing in kube-system. Deleting the namespace is a complete cleanup.

8. What you learned

* Probes are three different questions with three different answers. Liveness asks **should this container be restarted?**; readiness asks **should this Pod receive traffic?**; startup asks **has this container finished booting yet?** Confusing them causes outages in both directions - traffic sent to a cold replica (INC-4417) or a healthy replica killed forever (DEP-1012).

* Readiness is a traffic-control mechanism, not a health mechanism. You proved this by watching an address leave an EndpointSlice while RESTARTS stayed at 0, and by serving five successful requests through a degraded Deployment.

* Failure budgets are arithmetic, not vibes. $\text{initialDelaySeconds} + (\text{periodSeconds} \times \text{failureThreshold})$ is the number that matters; compare it against the application's real worst-case timing.

* The startup probe exists precisely so liveness can stay aggressive. Without it you must choose between slow detection of hangs and killing slow starters. With it you get both.

* EndpointSlice is the current API for Service backends; the v1 Endpoints object was deprecated in Kubernetes v1.33 and should not be your first diagnostic in new work.

Further reading

* Kubernetes - **Configure Liveness, Readiness and Startup Probes**:

<<https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>>

* Kubernetes - **Pod Lifecycle → Container probes**:

<<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes>>

* Kubernetes - **EndpointSlices**: <<https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>>

* Kubernetes blog - **Endpoints deprecation (v1.33)**:

<<https://kubernetes.io/blog/2025/04/24/endpoints-deprecation/>>

* CNCF - KCNA curriculum, **Cloud Native Architecture → Observability**: <<https://github.com/cncf/curriculum>>